SECTION 1 — SQL (Advanced Revision Notes)

Q1 — Latest Customer Record from 300M Rows

Window Function vs Snapshot Table

Core Concept

When retrieving the latest record per customer from a very large table (e.g., 300M rows), you must decide:

- Compute latest record at query time using `ROW_NUMBER()`
  OR
- Maintain a precomputed snapshot table.

Option 1 — Using Window Function at Query Time

```
SELECT *
FROM (
 SELECT *,
       ROW_NUMBER() OVER (
         PARTITION BY customer_id
         ORDER BY updated_timestamp DESC,
                ingestion_timestamp DESC
       ) AS rn
 FROM customers_raw
```

```
) t
WHERE rn = 1;
```

What This Does

- Groups rows by `customer_id`
- Sorts each group by latest timestamp
- Selects most recent record

Why This Becomes Expensive

On very large datasets:

- Sorting requires significant memory.
- If memory is insufficient → database spills to disk.
- Repeating this computation for dashboards increases cost.

Option 2 — Snapshot Table (Production Strategy)

Instead of sorting every time:

- Maintain a curated table with only the latest record.
- Update it incrementally during ingestion.
- Downstream queries become fast and stable.

When to Choose Which

Scenario	Recommended Approach
Ad-hoc analysis	Window function
Frequent dashboard joins	Snapshot table
SLA-based system	Snapshot table

Important Terms

- **Partition (Window Partition):** Logical grouping of rows inside window function.
- **Sort Spill:** When sorting exceeds memory and temporary disk usage occurs.
- **Snapshot Table:** Precomputed table storing latest state per key.

Q2 — Deterministic Deduplication with Tied Timestamps

Core Problem

Two rows may have identical `updated_timestamp`.

If sorting only by timestamp:

- Output may change between runs.
- Results become non-deterministic.

Why This Is Dangerous

- Audit reports become unstable.
- Dashboards may show inconsistent values.
- Hard to debug production discrepancies.

Correct Solution — Add Secondary Ordering

```
ROW_NUMBER() OVER (
 PARTITION BY customer_id
 ORDER BY updated_timestamp DESC,
          ingestion_timestamp DESC
)
```

Why This Works

If timestamps tie:

- Latest ingestion time determines final row.
- Output becomes deterministic.

Important Terms

- **Deterministic Result:** Same input → same output every time.
- **Tie-breaker Column:** Secondary column used for stable ordering.

Q3 — Revenue Correction for Late Refunds

Business Scenario

- January revenue reported.
- Refund for January order arrives in February.

How to correct January revenue without rewriting entire month repeatedly?

Core Concept

This is a **Late-Arriving Fact** problem.

Late-arriving fact = event logically belongs to past business period but arrives later.

Correct Strategy

- Keep original orders.
- Store refunds separately.
- Compute net revenue as:

Revenue = Orders – Refunds

Query Example

```
SELECT
 SUM(o.amount - COALESCE(r.refund_amount, 0)) AS net_revenue
FROM orders o
LEFT JOIN refunds r
 ON o.order_id = r.order_id
WHERE o.order_date >= '2024-01-01'
 AND o.order_date <  '2024-02-01';
```

Why This Is Production-Safe

- Preserves audit trail.
- Avoids full table recomputation.
- Supports correction-based accounting.

Important Terms

- **Late-arriving fact:** Delayed record belonging to earlier period.
- **Audit Trail:** Ability to trace financial adjustments.

Q4 — CTE Performance Degradation

Scenario

An 8-step CTE query became 3x slower after release.

Why This Happens

Possible reasons:

1. Intermediate datasets increased in size.
2. Filters applied too late.
3. Joining versioned tables before deduplication.
4. CTE materialization by query engine.

What Is Materialization?

Materialization = storing intermediate CTE result physically before continuing execution.

This increases:

- Disk IO
- Memory usage
- Execution time

Optimization Strategy

- Push filters early.
- Avoid `SELECT *`.
- Deduplicate dimension tables first.

- Pre-aggregate before joins.
- Stage heavy steps physically if reused.

Important Terms

- **Intermediate Dataset:** Output of one transformation used by next.
- **Materialization:** Temporary storage of query step output.

Q5 — DISTINCT Hiding Modeling Problems

Problem

Using DISTINCT to remove duplicates after a join.

Why This Is Dangerous

Duplicates usually indicate:

- Incorrect join condition.
- Joining to versioned dimension table.
- Missing snapshot logic.

DISTINCT hides the root cause and adds sorting overhead.

Correct Approach

Join against deduplicated snapshot instead of raw dimension.

Important Terms

- **Band-aid Fix:** Temporary solution that hides real problem.
- **Join Cardinality:** Number of rows produced by a join.

Q6 — Window Function Performance Bottlenecks

Scenario

Running cumulative revenue per product on 120M rows is slow.

Why It Happens

Window functions require:

- Partitioning
- Sorting
- Frame computation

Sorting large partitions causes:

- Memory pressure
- Disk spill
- Slow execution

Optimization Strategy

Pre-aggregate before applying window.

```
WITH daily AS (
 SELECT product_id, order_date, SUM(amount) AS daily_amt
 FROM orders
 GROUP BY product_id, order_date
)
SELECT
 product_id,
 order_date,
 SUM(daily_amt) OVER (
   PARTITION BY product_id
   ORDER BY order_date
   ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
 )
FROM daily;
```

Important Terms

- **Window Frame:** Range of rows considered in calculation.
- **Data Skew:** Uneven distribution of values across partitions.

Q7 — RANGE vs ROWS in Window Frames

Difference

- RANGE groups rows by ORDER BY value.
- ROWS operates row-by-row.

If multiple rows share same date:

- RANGE may include all at once.
- ROWS ensures strict cumulative behavior.

Always prefer ROWS for cumulative totals.

Q8 — Consecutive Login Streak Detection

Concept

Use LAG to compare current login_date with previous login_date.

If difference = 1 day → same streak
Else → new streak

This is called the **Gaps and Islands** problem.

Q9 — Top 5 Products Per Region with Stable Ranking

Requirement

- No skipped ranks
- Reproducible output

Correct Ranking Function

```
DENSE_RANK() OVER (
 PARTITION BY region
 ORDER BY revenue DESC, product_id ASC
)
```

DENSE_RANK avoids gaps.

Secondary ordering ensures reproducibility.

Q10 — When to Move from View to Curated Table

Move Logic to Physical Table When:

- Query is heavy.
- Query runs frequently.

- Performance SLA exists.
- Cost is high.

Views are good for abstraction, not heavy transformation.

Q11 — Why Indexes Are Ignored

Causes

- Applying function to indexed column.
- Datatype mismatch.
- Leading wildcard.
- Outdated statistics.

Sargable vs Non-Sargable

Sargable = can use index efficiently.

Bad:

```
WHERE YEAR(order_date) = 2024
```

Good:

```
WHERE order_date >= '2024-01-01'
 AND order_date <  '2025-01-01'
```

Q12 — Optimizing Join Between 10M and 500M Rows

Strategy

- Filter large table first.
- Join using snapshot dimension.
- Select minimal columns.
- Validate join cardinality.

Avoid join explosion.

Q13 — Partition Key Impact on Window Functions

Poor partition key choice leads to:

- Large partitions
- Sorting bottlenecks
- Uneven processing

Choose balanced partition keys.

Q14 — Handling Late-Arriving Facts

Use:

- Watermark tracking
- Rolling window reprocessing
- Incremental MERGE logic

Watermark = last processed timestamp boundary.

Q15 — SQL Debugging Process

Step-by-step debugging:

1. Run EXPLAIN plan.
2. Identify heavy scans/sorts.
3. Check join cardinality.
4. Verify early filtering.
5. Check window partition sizes.
6. Pre-aggregate if needed.

Production engineers debug logically, not randomly.

SECTION 2 — PYTHON

Q16 — You Receive a 7GB CSV Daily. How Do You Process It Without Memory Crash?

Core Concept

Pandas is an in-memory library.
A 7GB CSV file will likely consume **much more than 7GB in RAM** once loaded due to:

- Data type expansion
- Object overhead
- Temporary memory allocations

Loading it directly with `pd.read_csv()` can crash the system.

Why Memory Increases After Loading

- CSV is compressed text format.
- Pandas converts text into Python objects.
- Object dtype consumes large memory.
- Temporary buffers are created during parsing.

Safe Production Approach

1️⃣ Use Chunk Processing

import pandas as pd

```
chunksize = 200_000

for chunk in pd.read_csv("orders.csv", chunksize=chunksize):
    chunk = chunk[chunk["status"] == "completed"]
    process(chunk)
```

This:

- Reads limited rows at a time.
- Keeps memory usage stable.
- Avoids full dataset load.

2️⃣ Write Incrementally

Instead of storing processed data in memory:

- Write each chunk to Parquet/Delta.
- Append to database.

3️⃣ Move to Spark When Necessary

If:

- File size grows continuously.
- Complex joins required.
- SLA becomes strict.

Then Spark becomes the scalable solution.

Important Terms

- **Chunk Processing:** Reading data in batches.
- **Memory Spike:** Temporary increase in RAM usage.
- **SLA (Service Level Agreement):** Expected time for job completion.

Q17 — How Do You Design an Idempotent ETL Job?

Core Concept

Idempotent means:

Running the same job multiple times produces the same final result.

This is critical because:

- Jobs fail.
- Reruns happen.
- Retries are common.

Why Non-Idempotent Pipelines Are Dangerous

If you insert blindly:

- Duplicate records appear.
- Financial data becomes incorrect.
- Downstream metrics become unreliable.

Production Strategy

1️⃣ Use a Stable Primary Key

Example:

- `order_id`
- Or composite key `(order_id, source_system)`

2️⃣ Use Upsert Logic

Instead of:

INSERT INTO target

Use:

- Update existing records.
- Insert new records.

Conceptually:

If key exists → UPDATE
Else → INSERT

3️⃣ Use Batch Identifiers

Maintain:

- batch_id
- ingestion_timestamp

So reruns can be tracked and audited.

Important Terms

- **Idempotent:** Safe to rerun without changing final correctness.
- **Upsert:** Update + Insert operation.
- **Primary Key:** Unique identifier of a record.

Q18 — API Ingestion Fails After 10,000 Records. How Do You Resume?

Core Problem

APIs fail due to:

- Rate limits
- Network issues
- Server errors

Restarting from scratch is inefficient.

Correct Production Design

1️⃣ Implement Checkpointing

Store:

- Last successfully processed ID
- Or last timestamp

Example concept:

last_id = load_checkpoint()

After each successful page:

save_checkpoint(new_last_id)

If job fails:

- Restart from last checkpoint.
- Avoid reprocessing old records.

2️⃣ Implement Retry with Exponential Backoff

Instead of retrying immediately:

- Wait 1 second
- Then 2 seconds
- Then 4 seconds

Prevents overloading API.

Important Terms

- **Checkpoint:** Saved progress state.
- **Exponential Backoff:** Increasing retry delay after failures.
- **Offset:** Position marker in dataset.

Q19 — Why Are Python For-Loops a Performance Risk?

Core Concept

Python is an interpreted language.

Row-by-row loops:

- Execute in Python interpreter.
- Have high overhead.
- Are slow for large datasets.

Bad Pattern

```
for i in range(len(df)):
  df.loc[i, "new_col"] = df.loc[i, "amount"] * 1.1
```

This:

- Iterates one row at a time.
- Performs repeated indexing.
- Is extremely slow.

Better Pattern (Vectorization)

```
df["new_col"] = df["amount"] * 1.1
```

Vectorized operations:

- Use optimized C backend.
- Avoid Python-level loop overhead.
- Are significantly faster.

Important Terms

- **Vectorization:** Applying operations to entire column at once.
- **Interpreter Overhead:** Extra time spent running Python execution logic.

Q20 — How Do You Handle Corrupted or Malformed Records?

Core Principle

Do not stop entire pipeline because of few bad records.

Production systems must:

- Continue processing valid data.
- Isolate invalid records.

Correct Strategy

1. Validate critical fields.
2. Separate invalid rows into quarantine table.
3. Log error reason.
4. Continue pipeline execution.

Example Validation Logic

```
valid = df[df["amount"] >= 0]
invalid = df[df["amount"] < 0]

write_to_valid_table(valid)
write_to_quarantine(invalid)
```

Why This Is Important

- Prevents data loss.
- Supports audit and debugging.
- Maintains pipeline availability.

Important Terms

- **Quarantine Table:** Storage for invalid records.
- **Validation Rule:** Condition data must satisfy.
- **Data Integrity:** Accuracy and consistency of data.

Q21 — When Should You Use Multiprocessing?

Core Concept

Multiprocessing is useful when:

- Workload is CPU-bound.
- Heavy transformations or calculations exist.

It is NOT very useful when:

- Workload is IO-bound.
- Bottleneck is network or disk.

CPU-Bound vs IO-Bound

Type	Meaning
CPU-bound	Limited by processor computation
IO-bound	Limited by disk or network waiting

Example Use Case

- Complex feature engineering
- Large JSON transformations
- Encryption/compression

Important Terms

- **Parallelism:** Running tasks simultaneously.
- **CPU-bound:** Limited by computation speed.
- **IO-bound:** Limited by waiting time.

Q22 — Logging and Monitoring in Production ETL

Core Concept

Without logging, debugging becomes guesswork.

Production ETL must log:

- Job start time
- Job end time
- Rows processed
- Errors count
- Duration
- Batch ID

Basic Logging Example

```
import logging
logging.basicConfig(level=logging.INFO)
```

```
logging.info("Job started")
logging.info("Rows processed: 200000")
logging.info("Job finished")
```

Monitoring Should Detect

- Sudden drop in row count
- Unusual runtime increase
- Error spikes
- Late data arrival

Important Terms

- **Observability:** Ability to understand system behavior from logs.
- **Monitoring:** Continuous tracking of system health.
- **Alerting:** Notifying when thresholds are breached.

Q23 — When Is Python No Longer the Right Tool?

Python Is Not Suitable When:

1. Dataset consistently exceeds RAM.

2. Job runtime exceeds SLA.
3. Heavy joins required.
4. Data volume grows continuously.
5. Fault tolerance required.

Why Spark Becomes Necessary

Spark provides:

- Distributed processing
- Fault tolerance
- Horizontal scaling
- Shuffle-based large joins

Key Insight

Chunking is a temporary solution.
Distributed compute is a scaling strategy.

Important Terms

- **Horizontal Scaling:** Adding more machines.
- **Vertical Scaling:** Increasing machine RAM/CPU.
- **Fault Tolerance:** System survives failure.

SECTION 3 — PANDAS

Q24 — Why Does Merging Two 20M Row DataFrames Often Crash Memory?

Core Concept

Pandas operates completely in memory on a single machine.
When performing a merge, memory usage can temporarily increase significantly.

This often causes crashes when working with large datasets (e.g., 20M+ rows).

What Happens Internally During Merge

When executing:

merged = df1.merge(df2, on="customer_id")

Pandas:

1. Keeps both DataFrames fully in memory.
2. Creates hash maps for join keys.
3. Allocates intermediate join arrays.
4. Builds a new output DataFrame.

This can temporarily increase memory usage to 2–3× total input size.

Why Memory Explodes

1️⃣ Both Inputs Stay in Memory

Pandas does not discard original tables during merge.

2️⃣ Temporary Join Structures

Hash maps and indexing arrays require additional RAM.

3️⃣ Row Explosion (Many-to-Many Joins)

If join key contains duplicates on both sides:

Rows multiply.

Example:

- 1 order joins to 3 customer versions
  → 3 output rows created

Production Optimization Strategy

1️⃣ Select Only Required Columns

```
df1 = df1[["order_id", "customer_id", "amount"]]
df2 = df2[["customer_id", "city"]]
```

Reducing columns directly reduces memory usage.

2️⃣ Optimize Data Types

```
df1["customer_id"] = df1["customer_id"].astype("int32")
df2["city"] = df2["city"].astype("category")
```

- `int32` uses half memory of `int64`.
- `category` compresses repeated string values.

3️⃣ Validate Join Cardinality

Check duplicates before merge:

df2["customer_id"].value_counts().head()

If duplicates exist, deduplicate first.

4️⃣ Move to Spark When Data Size Grows

If dataset consistently approaches memory limit, switch to distributed processing.

Important Terms

- **Cardinality:** Number of unique values in a column.
- **Row Explosion:** Increase in rows due to duplicate join keys.
- **In-memory Engine:** System that requires entire dataset in RAM.

Q25 — How Do You Optimize a Slow `groupby()` Operation?

Core Concept

`groupby()` uses hash-based grouping internally.

Performance depends on:

- Data size
- Key cardinality
- Data types

- Sorting requirements

Why `groupby()` Becomes Slow

1️⃣ Large Dataset

More rows → more hashing work.

2️⃣ High Cardinality Keys

If grouping by nearly unique values, grouping becomes expensive.

3️⃣ Object (String) Columns

Strings are slower to hash than integers.

4️⃣ Sorting Enabled

Sorting grouped output adds extra cost.

Optimization Strategy

Step 1 — Filter Early

```
df = df[df["order_date"] >= "2024-01-01"]
```

Always reduce dataset before grouping.

Step 2 — Select Only Required Columns

```
df = df[["customer_id", "amount"]]
```

Avoid grouping unnecessary columns.

Step 3 — Optimize Data Types

```
df["customer_id"] = df["customer_id"].astype("int32")
```

Smaller numeric types improve hashing performance.

Step 4 — Disable Sorting If Not Required

```
df.groupby("customer_id", sort=False)["amount"].sum()
```

Sorting increases computation time.

Production Insight

Performance of groupby roughly scales with:

Data Volume × Key Complexity

Important Terms

- **Hash Grouping:** Grouping using internal hash tables.
- **Aggregation:** Reducing multiple rows into summary value.
- **High Cardinality:** Many unique values.

Q26 — How Does Data Type Optimization Reduce Memory Usage?

Core Concept

Default data types in Pandas are often larger than necessary.

Optimizing data types can:

- Reduce memory usage by 30–60%.
- Improve performance.
- Prevent crashes.

Numeric Type Optimization

Example:

- `int64` → 8 bytes per row
- `int32` → 4 bytes per row

For 20M rows:

- int64 = ~160MB
- int32 = ~80MB

Memory reduced by half.

String Optimization Using `category`

If a column like `city` contains repeated values:

Object dtype:

- Stores each string separately.

Category dtype:

- Stores unique values once.

- Replaces each row with integer code.

Example:

df["city"] = df["city"].astype("category")

This dramatically reduces memory usage.

Measuring Memory Usage

df.memory_usage(deep=True).sum()

Use `deep=True` to measure actual memory used by string objects.

Why This Matters in Production

Lower memory usage means:

- Faster joins
- Faster groupby
- Stable execution
- Reduced system crashes

Important Terms

- **Memory Footprint:** Total memory consumed.
- **Category Encoding:** Dictionary-based storage of repeated values.
- **Object dtype:** Default string storage type in Pandas.

Q27 — When Should Pandas NOT Be Used?

Core Concept

Pandas is excellent for:

- Small to medium datasets
- Exploratory data analysis
- Feature engineering
- Prototyping

But it has scaling limitations.

Do Not Use Pandas When:

1️⃣ Data Consistently Exceeds RAM

Chunking may help temporarily, but performance degrades over time.

2️⃣ Heavy Joins on Large Tables Required

Single-machine execution becomes bottleneck.

3️⃣ SLA Requires Predictable Runtime

Single node scaling is limited.

4️⃣ Fault Tolerance Is Required

If job crashes halfway, Pandas does not provide automatic distributed recovery.

When to Switch to Spark

Switch when:

- Data growth is continuous.
- Job runtime increases each month.
- Memory tuning becomes repetitive effort.
- Distributed compute is necessary.

Comparison Table

Feature	Pandas	Spark
Execution	Single machine	Distributed
Scaling	Vertical only	Horizontal
Fault tolerance	Limited	Built-in
Suitable for	Medium data	Large data

Important Terms

- **Horizontal Scaling:** Adding more machines.
- **Vertical Scaling:** Increasing RAM/CPU on single machine.
- **Distributed Processing:** Computation across multiple nodes.

SECTION 4 — PYSPARK (Advanced Revision Notes)

Q28 — One Product Accounts for 60% of Orders. Why Does Spark Job Slow Down?

Core Concept

This is a **data skew problem**.

Data skew occurs when one key has significantly more records than others.

Why This Slows Down Spark

Spark distributes data across partitions.

If:

- Product A = 60% of rows
- Other products = remaining 40%

Then:

- One partition becomes extremely large.
- Other partitions finish quickly.
- Entire job waits for slow partition.

This causes:

- Uneven task distribution.
- Longer stage execution.
- Possible memory spill.

How to Detect Data Skew

df.groupBy("product_id").count().orderBy("count", ascending=False).show()

If one key dominates, skew exists.

Solutions

1️⃣ Salting Technique

Add random suffix to heavy key:

```
from pyspark.sql.functions import rand

df = df.withColumn("salt", (rand()*10).cast("int"))
df = df.withColumn("product_salted", concat("product_id","salt"))
```

This distributes heavy key across partitions.

2️⃣ Pre-Aggregation

Aggregate before heavy joins.

Important Terms

- **Data Skew:** Uneven distribution of data across partitions.

- **Partition:** Logical data chunk processed by Spark task.
- **Salting:** Artificially spreading skewed keys.

Q29 — When Does Broadcast Join Become Dangerous?

Core Concept

Broadcast join copies small table to all worker nodes.

It avoids shuffle.

Why It Can Be Dangerous

Broadcasting works only when:

- Table is truly small.
- Fits in executor memory.

If table grows:

- Memory pressure increases.
- Executors may crash.
- Job fails unpredictably.

When to Use Broadcast

- Dimension table < few hundred MB.
- Rare updates.
- Stable size.

When NOT to Use

- Table size growing over time.
- Executor memory already high.
- Join involves multiple large tables.

Important Terms

- **Broadcast Join:** Replicating small dataset to all nodes.
- **Shuffle:** Data redistribution across nodes.
- **Executor:** Worker process in Spark cluster.

Q30 — Why Are Too Many Small Files Harmful?

Core Concept

This is the **small files problem**.

Distributed storage systems (HDFS, S3, ADLS) handle large files efficiently. Many small files create overhead.

Why Small Files Are Bad

- Each file has metadata overhead.
- Spark creates one task per file.
- Scheduler overhead increases.
- Performance degrades.

Example

10,000 small files = 10,000 tasks
More overhead, slower execution.

Solution

1 Repartition Before Write

```
df.repartition(100).write.parquet("path")
```

2 Compaction

Merge small files into larger ones.

Important Terms

- **Metadata Overhead:** System cost of managing file information.

- **Compaction:** Merging small files into fewer large files.

Q31 — When Should You NOT Cache in Spark?

Core Concept

Caching stores dataset in memory.

It improves performance only when reused multiple times.

Why Caching Can Be Harmful

- Consumes executor memory.
- Causes memory pressure.
- May trigger garbage collection.
- May cause spill to disk.

Correct Use of Cache

Cache when:

- Data reused multiple times.
- Expensive computation.
- Fits comfortably in memory.

Remove Cache When Done

df.unpersist()

Important Terms

- **Cache:** Store dataset in memory.
- **Garbage Collection:** Memory cleanup process.
- **Spill:** Writing memory overflow to disk.

Q32 — How Does Repartition Impact Shuffle?

Core Concept

Repartition triggers shuffle.

Shuffle = redistribution of data across cluster nodes.

Why Shuffle Is Expensive

Shuffle requires:

- Disk IO
- Network transfer

- Serialization

It is one of the most expensive Spark operations.

When to Use Repartition

- Before writing output.
- Before join to balance partitions.
- To increase parallelism.

Avoid Unnecessary Repartition

Multiple repartitions increase shuffle cost.

Important Terms

- **Shuffle:** Network-based data redistribution.
- **Serialization:** Converting data into transferable format.
- **Parallelism:** Number of tasks running simultaneously.

Q33 — How to Design Incremental Loading Using Spark and Delta?

Core Concept

Incremental loading avoids full table rewrite.

Strategy

1. Identify new/updated records.
2. Merge into existing table.
3. Maintain watermark.

Example MERGE

```
deltaTable.alias("target") \
 .merge(
   source.alias("source"),
   "target.order_id = source.order_id"
 ) \
 .whenMatchedUpdateAll() \
 .whenNotMatchedInsertAll() \
 .execute()
```

Why This Is Powerful

- Avoids full table overwrite.
- Preserves history.
- Supports late-arriving data.

Important Terms

- **MERGE:** Upsert operation.
- **Watermark:** Last processed timestamp.
- **Incremental Load:** Process only new data.

Q34 — Wide vs Narrow Transformations

Narrow Transformation

- No shuffle.
- Data stays in same partition.

Examples:

- map
- filter

Wide Transformation

- Requires shuffle.
- Data redistributed across nodes.

Examples:

- groupBy
- join

- distinct

Why This Matters

Wide transformations are expensive.

Always try to minimize shuffle.

Important Terms

- **Stage:** Spark execution phase.
- **DAG (Directed Acyclic Graph):** Execution plan graph.

Q35 — How to Detect and Handle Data Skew?

Detection

- Spark UI → Stage view
- Long-running single task
- Uneven task completion time

Solutions

- Salting
- Pre-aggregation
- Increase shuffle partitions
- Use skew join hints

Important Terms

- **Spark UI:** Monitoring interface.
- **Skew Join:** Join affected by skewed key distribution.

Q36 — How to Tune Spark for Production?

Key Tuning Areas

1. Partition count
2. Executor memory
3. Shuffle partitions
4. Join strategy
5. File size optimization

General Best Practices

- Avoid small files.
- Monitor Spark UI.
- Reduce shuffle.
- Cache only when reused.

SECTION 5 — FINAL DESIGN ROUND (Q37–Q40)

Q37 — Design Hourly Revenue Pipeline with Late Refund Handling

---

Architecture Layers

Bronze → Raw ingestion
Silver → Cleaned and deduplicated
Gold → Business aggregates

Key Requirements

- Idempotent ingestion
- Late refund adjustment

- Incremental merge
- Dashboard-ready tables

Design Flow

1. Ingest raw orders and refunds.
2. Deduplicate using window logic.
3. Merge incremental updates.
4. Compute net revenue in Gold layer.
5. Maintain watermark.

Production Considerations

- SLA tracking
- Data validation
- Partitioning by date
- Monitoring alerts

Q38 — How SQL, Python, and Spark Decisions Interact

Roles in Production

- Python → Orchestration and API ingestion.
- Spark → Heavy transformations.
- SQL → Business aggregation and analytics.

Tool Selection Principle

Use tool based on:

- Scale
- Latency
- Complexity
- Cost

Q39 — Trade-offs: Real-Time vs Batch

Batch

- Simpler
- Cheaper
- Easier to maintain
- Suitable for daily reporting

Real-Time

- Low latency
- Higher complexity
- Higher cost
- Operationally demanding

Decision Factors

- Business urgency
- Cost tolerance
- Engineering complexity

Q40 — What Makes an Interviewer Confident You Are Production-Ready?

Strong Signals

- You talk about trade-offs.
- You mention scaling limits.
- You understand skew and shuffle.
- You design idempotent systems.

- You discuss monitoring.
- You explain debugging steps logically.